

Relatório Microprocessadores - Projeto 1

Disciplina: Microcontroladores
2025.2

Equipe:

Marina Vasques Rodrigues

E-mail: marinavasq18@alu.ufc.br

Matrícula: 496500

Iuri Fernandes Bastos

E-mail: iuriferbastos@gmail.com

Matrícula: 563536

Gustavo Oliveira Seabra

E-mail: gustavooliveiraseabra@gmail.com

Matrícula: 567464

Jonas Albuquerque Araujo

E-mail: jonasalbuquerque@alu.ufc.br

Matrícula: 404213

João Otavio Brasil Nascimento

E-mail: otavio.brasil@alu.ufc.br

Matrícula: 563537

1 Introdução

O projeto desenvolvido tem o objetivo de introduzir aos fundamentos de arquitetura e funcionamento de um microprocessador. O projeto implementa, em VHDL, módulos básicos responsáveis pela leitura e execução de instruções, simulando o comportamento de uma unidade de processamento.

2 Requisitos do Compilador de Assembly

- Identificação e remoção de comentários.
- Implementação do conjunto mínimo de instruções.
- Análise semântica para palavras inválidas.
- Análise sintática: verificar número correto de operandos por instrução.

3 Compilador

Nessa seção, explicaremos o funcionamento e como usar o compilador:

3.1 Mapeamento de Códigos

O compilador utiliza uma organização de código estrutural, contendo a main que rege o funcionamento principal e algumas funções derivadas. as funções e utilitários são:

1. **main** - função principal, desenvolve a lógica de leitura e escrita do código chamando as demais funções.
2. **identar** - retira comentarios e espaços iniciais do código
3. **pegar_ppalavra** – *obtem a proxima palavra do código para analisarmos.*
3. **obter_oopcode** – *obtem o opcode a partir do mnemônico fornecido.*
3. **obter_dado** – *verifica se o operando é binário, hexadecimal ou decimal e chama alguma das funções abaixo.*
3. **trad_hexa** – *traduz o operando para hexadecimal para binário.*
3. **trad_bin** – *identifica o operando e se é binário.*
3. **trad_dec** – *traduz o operando para decimal para binário.*
3. **erro** - acionada em caso de erro, interrompe o processo;

3.2 Estrutura do assembly

o assembly criado no nosso projeto segue a logica de:

- Cada instrução utiliza 1 mnemônico e de 0 a 2 operandos.
- Cada instrução ocupa uma linha, com uma instrução por linha.
- A sequencia deve seguir, primeiro o mnemônico e depois os operandos, na mesma linha e em seguida.
- Os mnenomios devem estar em letra maiúscula.
- comentarios são feitos com "//".
- os parametros podem ser em hexadecimal, decimal ou binario. Para tanto, utilize os prefixos "0x", "0d", "0b" respectivamente.
- os parametros não podem passar do valor 248, valor maximo permitido.

As instruções possíveis são as seguintes:

Table 1: Resumo das instruções do conjunto mostrado

Mnemo.	opcode(Bin)	Formato (operandos)	Efeito / Descrição
MOV	00000000	MOV <valor> <end>	Escreve o <i>valor</i> no <i>endereço</i> de memória indicado.
ADD	00000001	ADD <endA> <endB>	Soma os dados em <endA> e <endB>.
ADI	00000010	ADI <valor>, <end>	Soma um imediato (<valor>) ao dado no <end>.
NOP	11111111	NOP	Não realiza operação (consome um ciclo).
SUB	0000011	SUB <endA> <endB>	Subtrai o valor de <endB> do valor em <endA>.
MUL	00000101	MUL <endA> <endB>	Multiplica os dados dos endereços <endA> e <endB>.
DIV	00000100	DIV <endA> <endB>	Divide o valor em <endA> pelo valor em <endB>.
RES	00000110	RES <endA> <endB>	Calcula o resto da divisão.
CMP	00000111	CMP <endA> <endB>	Compara os dados em <endA> e <endB>.
AND	00001000	AND <endA> <endB>	Executa operação bit a bit AND.
ORR	00001001	ORR <endA> <endB>	Executa operação bit a bit OR.
NOT	00001010	NOT <end>	Inverte todos os bits do dado no endereço indicado.
XOR	00001011	XOR <endA> <endB>	Executa operação bit a bit XOR.

3.3 Funcionamento do Compilador

Tendo um arquivo "assembly.txt", com os dados seguindo a estrutura acima. O compilador abre esse arquivo para a leitura e criará um outro arquivo "dados.bin".

Para cada linha, o compilador fará os seguintes passos:

- Identifica a linha, retirando os comentários e os espaços iniciais;
- Caso não sobre nada na linha, significa que a linha não contém uma instrução, então pula para a próxima.
- Caso ache uma instrução, ele obtém o mnemônico correspondente.
- Com o mnemônico, traduzimos para o opcode correspondente, escrevemos e pegamos a quantidade de parâmetros.
- Para a quantidade de parâmetros, o compilador lê os operandos e traduz para binário, escrevendo no arquivo.
- caso ocorra algum erro, o compilador aciona a função de erro.

3.4 Mensagens de Erro

Quando uma ocasião de erro é acionada, o compilador exclui o arquivo binário e imprime a mensagem de erro.

Todos os casos seguem a estrutura:

- **ERROR – line: <numero da linha>: <mensagem de erro> <palavra errada>**
- **Excluindo arquivo**

Temos no total 5 possibilidades de mensagens de erro, comportando cada caso:

1. **"arquivo nao aberto"** - acionada caso não ache o arquivo assembly.txt.
2. **"command not found"** - acionado quando o mnemônico não é achado na tabela de mnemonicos.
3. **"missing arguments"** - acionado quando temos menos parâmetros que o requisitado para a instrução;
4. **"argument invalid:"** - acionado quando o parâmetro da instrução não está certo. seja por erro de digitação ou por que o dado ultrapassou 8 bits.
5. **"most arguments"** - acionado quando temos mais parâmetros que os necessários para mnemônico.

4 Requisitos do Processador

- Palavra de pelo menos 8 bits.
- Registradores obrigatórios: 3 para operações matemáticas e 1 para flags.
- Conjunto mínimo de instruções:
 - Movimentação e transferência de dados
 - Manipulação de bits e deslocamento
 - Operações aritméticas: ADD, SUB, MUL, DIV, MOD
 - Operações relacionais: comparação
 - Operações lógicas: AND, OR, NOT, XOR

5 Processador

Este processador foi projetado para realizar operações aritméticas e lógicas básicas, além de implementar a funcionalidade de transferência de dados (MOV), que copia valores entre operandos diretamente no processador. O design é baseado em uma arquitetura simplificada, conforme detalhado nos módulos VHDL fornecidos (alu-8-bit.vhd, register-bank-simplified.vhd, decoder-controller-simplified.vhd e processor-core-simplified.vhd).

O processador se comunica com uma memória de instruções externa, que armazena todas as instruções a serem executadas. Cada instrução é organizada em um formato de 24 bits, dividido em:

- Opcode (8 bits): Define a operação a ser realizada.
- Operand1 (8 bits): Geralmente o primeiro operando ou o endereço de destino/origem.
- Operand2 (8 bits): Geralmente o segundo operando ou um dado imediato.

O núcleo do processador é composto por três principais componentes interconectados: o Controller (Decodificador/Controlador), a ULA (Unidade Lógica e Aritmética) e o módulo principal Processor (processor-core-simplified).

Componentes do Processador

- Controller (Decodificador/Controlador): O decoder-controller-simplified é responsável por decodificar o opcode recebido e determinar a operação correspondente. Sua função principal é gerar os sinais de controle específicos para cada tipo de operação, direcionando o fluxo de dados entre os demais componentes.
- ULA (Unidade Lógica e Aritmética): A alu-8-bit é o componente executivo que realiza as operações aritméticas e lógicas. É uma ULA de 8 bits que opera em conjunto com um registrador de saída síncrono.
- Módulo Principal (Processor Core): O módulo principal (processor-core-simplified) integra o Controller, a ULA e o Banco de Registradores, coordenando o fluxo de dados entre eles.
 - Lê as instruções (simuladas pelas entradas opcode-in, data-A-in, data-B-in).
 - Decodifica e executa cada instrução baseada nos sinais de controle do Controller.
 - Coordena a comunicação com a ULA para operações aritméticas e lógicas.
 - Realiza diretamente a operação MOV (transferência de dado imediato para registrador).
 - Sincroniza o processamento com o sinal de clock (clk) e o sinal de reset.

Ciclo de Operação

O processador opera de forma síncrona, seguindo um ciclo de execução de instrução simplificado. O ciclo completo envolve a busca (simulada pela entrada de opcode e dados), decodificação e execução.

- 1.Busca e Decodificação: O opcode, operand1 e operand2 da instrução são lidos (simulados pelas entradas). O Controller decodifica o opcode e gera os sinais de controle.
- 2.Preparação dos Dados: O Banco de Registradores, se necessário, lê os operandos A e B com base nos endereços fornecidos pelo Controller.
- 3.Execução (ULA): Se a instrução requer o uso da ULA (ex: ADD-REG), os dados são enviados e a ULA processa a operação, gerando o resultado e as flags. O resultado da ULA é registrado síncronamente.
- 4.Execução (Não-ULA): Caso o opcode seja de MOV (ex: OP-IMM), o dado imediato é copiado diretamente para o registrador de destino, sem passar pela ULA.
- 5.Escrita de Volta: O resultado da ULA ou o dado da operação MOV é escrito de volta no Banco de Registradores, conforme o endereço e o sinal de habilitação de escrita (reg-write-en) definidos pelo Controller.
- 6.Sinais de Saída: O resultado e as flags são atualizados e disponibilizados nas saídas do módulo principal.

Operações Suportadas

O processador suporta um conjunto de instruções que podem ser categorizadas em operações de transferência de dados e operações de processamento (ULA).

- Aritméticas: ADD, SUB, MUL, DIV, MOD (Módulo).
- Lógicas: AND, OR, NOT, XOR.
- Comparação: CMP (Compara os operandos, afetando as flags).

Sinais de Saída

Após a execução de cada instrução, o estado do processador é refletido através de sinais de saída, que incluem o resultado da operação e um conjunto de flags (bandeiras de estado).

- result (ula-result-out): Contém o resultado da operação executada pela ULA. No caso de operações MOV que não passam pela ULA, o dado é escrito diretamente no registrador, e a saída da ULA pode não ser relevante ou estar desabilitada.
- flags (ula-cout-out, ula-zero-flag-out): Indicam o estado do processador após a execução da instrução.
 - Z (Zero Flag): Indica se o resultado da operação é zero.
 - C (Carry Flag): Indica carry (transporte) ou borrow (empréstimo), gerado em operações aritméticas.

6 Memória de Instruções

6.1 Organização e Estrutura

A memória de instruções (*InstructionMemory*) é o componente responsável por armazenar o conjunto de instruções que serão executadas pelo processador. Ela é organizada em palavras de 24 bits e possui 256 posições endereçáveis, permitindo a leitura e escrita de instruções por meio de um sinal de controle.

Cada instrução é composta por três campos principais: **opcode**, **data_A** e **data_B**. Essa estrutura padronizada facilita a decodificação e o envio das informações para a Unidade Lógica e Aritmética (ULA) e para os demais módulos do sistema.

6.2 Formato das Instruções

Cada palavra de 24 bits é segmentada nos seguintes campos:

- **Campo opcode (bits 23 a 16):** Define a operação a ser executada. É utilizado pela unidade de controle para determinar o tipo de processamento realizado pela ULA.
- **Campo data_A (bits 15 a 8):** Representa o primeiro operando da instrução. Pode conter valores imediatos, endereços de registradores ou dados de entrada.
- **Campo data_B (bits 7 a 0):** Contém o segundo operando, utilizado em operações aritméticas e lógicas que requerem dois valores, como soma, subtração ou comparação.

6.3 Funcionamento Geral

O processo de leitura e escrita é controlado pelo sinal de *clock* (**clk**) e pelo sinal de habilitação de escrita (**write_enable**). Quando **write_enable** está em nível lógico alto, o dado presente em **data_in** é armazenado na posição de memória indicada por **address**. A cada borda de subida do **clk**, o conteúdo dessa posição é disponibilizado na saída **data_out**, permitindo que o processador acesse a instrução correspondente.

7 Sistema de Leitura do Arquivo Binário

7.1 Arquitetura de Carregamento

O processo de inicialização do sistema é responsável por ler o conteúdo do arquivo binário *dados.bin* e convertê-lo em instruções compreensíveis pelo processador. Essa função é executada pelo módulo *InstructionLoader*, que organiza os dados lidos e os grava na *InstructionMemory* para posterior execução.

7.2 Módulo Carregador de Instruções

O *InstructionLoader* atua como ponte entre o armazenamento externo e a memória de instruções. Na inicialização, ele lê o arquivo *dados.bin*, converte cada linha em um vetor binário de 8 bits e armazena o conteúdo em uma memória interna de 256 posições. A partir desses dados, o módulo monta instruções de 24 bits — compostas por um opcode e dois operandos — e as transfere para a memória externa (*InstructionMemory*) de forma sequencial.

7.2.1 Máquina de Estados

O carregamento segue uma máquina de estados finitos que garante a leitura e montagem ordenada das instruções:

1. **LOAD_OPCODE**: Lê o opcode (8 bits) da memória interna.
2. **LOAD_DATA_A**: Lê o primeiro operando da instrução.
3. **LOAD_DATA_B**: Lê o segundo operando.
4. **WRITE_MEMORY**: Combina os três campos (opcode, operando A e operando B) e grava o resultado na *InstructionMemory*.
5. **OUTPUT_READY**: Sinaliza que a instrução foi carregada com sucesso e está pronta.
6. **DONE_STATE**: Indica o término do processo de carregamento.

O módulo também gera mensagens de depuração via comandos **report**, informando o estado atual e os dados carregados em cada etapa, facilitando a análise durante simulação.

7.3 Módulo de Controle Principal

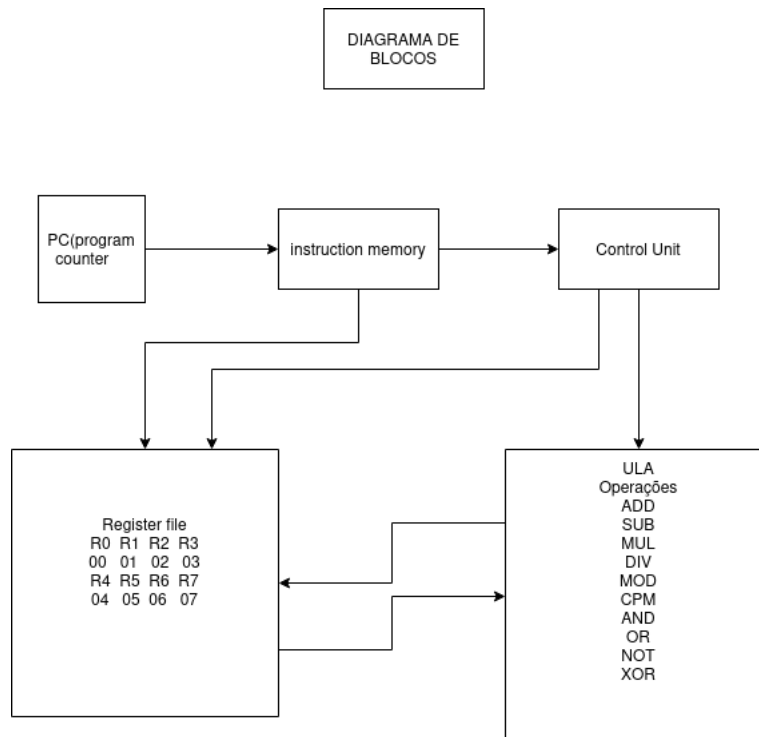
O módulo *TopLevel* coordena a operação do sistema, integrando o *InstructionLoader*, a *InstructionMemory* e o *processor_core_simplified*. Ele controla duas fases principais:

- **Carregamento**: o *InstructionLoader* grava as instruções na memória até o sinal *loader_done_sig* indicar que o processo foi concluído.
- **Execução**: o processador lê as instruções da memória e as executa sequencialmente, enquanto o contador de programa (*program_counter*) é incrementado a cada ciclo.

O processo *SYSTEM_CONTROL* gerencia a transição entre as fases e o sinal *done_signal*, que indica o término da execução após 10 instruções. Um segundo processo, *MONITOR*, exibe no console o andamento da execução, mostrando o endereço atual, a instrução e o resultado da ULA para fins de depuração.

8 Diagramas e Tabelas

8.1 Diagrama de Blocos



8.2 Tabela de Registradores e Opcodes

Table 2: Tabela de instruções e opcodes

Opcode	Instrução	Operação	Tipo
00	MOV	Transfere dado para endereço	Dados
01	ADD	Soma valores de dois endereços	Aritmética
02	ADDin	Soma valor imediato e endereço	Aritmética
03	SUB	Subtração entre endereços	Aritmética
04	DIV	Divisão entre endereços	Aritmética
05	MUL	Multiplicação entre endereços	Aritmética
06	RES	Resto da divisão entre endereços	Aritmética
07	CMP	Comparação entre endereços	Lógica
08	AND	Operação lógica “E”	Lógica
09	ORR	Operação lógica “OU”	Lógica
0A	NOT	Operação lógica “Negado”	Lógica
0B	XOR	Operação lógica “Ou exclusivo”	Lógica
FF	NOP	Nenhuma operação	Controle

Table 3: Tabela de Registradores

Registrador	Sinal de Entrada	Descrição
r1	operando1	Armazena o primeiro operando utilizado nas operações da ULA
r2	operando2	Armazena o segundo operando utilizado nas operações da ULA
r3	resultado	Armazena o resultado das operações realizadas pela ULA
r4	flags	Armazena os flags de status (zero, carry, negativo, etc.)